A Modular Framework for the Integration of Heterogeneous Notations and Tools

Robert Büssow and Wolfgang Grieskamp

Technische Universität Berlin, Institut für Kommunikations- und Softwaretechnik
Sekr. 5–6, Franklinstr. 28/29, D–10587 Berlin.
{buessow, wg}@cs.tu-berlin.de

Abstract. We report on the notation and tool integration framework developed in the application-oriented research project Espress. The framework is based on a Z core and provides means of integrating formalisms on a syntactical, semantical as well as tool technical level. Currently, the framework integrates Z with the languages of Statemate [15], i.e. Statecharts to describe reactive behavior and activity charts to describe structural aspects. Our means of integration are, however, general enough to integrate other, esp. synchronous languages, as well, and reuse much of the existing tools in the process.

1 Introduction

Every software-based system potentially benefits from the application of formal techniques. For the development of mission or even safety-critical embedded systems, their use is of particular advantage, because the potential damage operators and developers have to envisage in case of malfunction may be much worse than the additional costs of applying formal techniques.

However, the acceptance of formal techniques in industry is still relatively low. One reason probably is that most formal approaches require users to forget about their conventional techniques and tools, and to become acquainted in a new environment.

Based on this observation the Espress project was launched in 1995.[1] The objective of this project was to provide a framework for specifying *embedded systems* in which formal methods can be used in *combination* with existing, established commercial techniques and tools. The choice of the notations to be used was mainly determined by the needs of the industrial partners. For the description of the reactive behavior, Statecharts (i.e. the Statemate tool [13]) were chosen, and combined with the Z specification formalism [26, 27] for the description of data and temporal logic for the specification of safety-critical behavioral properties [23, 6].

In this paper, we report on the notation and tool integration framework developed in Espress. We present the basic design of the formalism $\mu\mathcal{SZ}$ and its semantics. Since integration of languages and integration of their associated tools are intrinsically interdependent, we also sketch the basic design principles of the tool environment ZeTa. In fact, the design of ZeTa [7] motivated a revision of earlier versions of $\mu\mathcal{SZ}$ [5] to increase *modularity* and *genericity*. Building tools and tool environments is a significant effort. Therefore, inventing appropriate abstractions may be of more interest in that context than for a mere pencil-and-paper definition of an integrated semantics.

[1] The Espress project is a cooperation of industry (Daimler-Benz AG and Robert Bosch AG) and academic (GMD-First, FhG-ISST, and TU-Berlin) research institutes, funded by the German Bundesministerium für Bildung, Wissenschaft, Forschung und Technologie.

The setting of $\mu\mathcal{SZ}$ presented in this paper is based on a *core* which is independent of notations such as Statemate Statecharts or temporal logic as used for behavioral specification in ESPRESS. The only appointments made are that

- we base $\mu\mathcal{SZ}$ on the formal notation Z (strictly speaken, Z needs not to be visible to the user – it at least constitutes the foundation);
- we adapt the *synchronous* approach to concurrent behavior by assuming a global step-wise execution model where concurrent activities are performed at once in a step (in contrast to interleaving). We use a *trace-based* semantics (e.g. [23]) to model this.

External notations are integrated by *plugins* which provide a *syntactic translation* into the $\mu\mathcal{SZ}$ core. This design leads to a large conformance between semantic and tool-technical integration, and enables a well-founded reuse of generic tools such as provers, model-checkers, and compilers, working on the core of $\mu\mathcal{SZ}$, for other notations.

The remaining parts of this paper are organized as follows. In Sect. 2, we sketch by "look and feel" the usage of our running framework in the context of ESPRESS. Section 3 describes the generic core of the $\mu\mathcal{SZ}$ notation, and its semantics. As an example for *plugins*, Sect. 4 presents the embedding of Statemate languages into the $\mu\mathcal{SZ}$ core. Sec. 5 sketches the design of the ZETA tool environment, and how it is intervened with the $\mu\mathcal{SZ}$ language. In Sec. 6 we discuss related work, and in Sec. 7 we present some conclusions.

2 Application

In this section we give a "look-and-feel" sketch of the application of the $\mu\mathcal{SZ}$ notation and the ZETA tool environment. More comprehensive discussions and case studies applying our framework can be found in [9, 11, 4, 7, 28].

2.1 A Specification Example

Figure 1 shows on example of a $\mu\mathcal{SZ}$ specification taken from one of the ESPRESS case-studies, an *intelligent cruise control* (ICC) [11]. In addition to a common cruise control, the intelligent cruise control has sensors to measure the distance to the traffic ahead. If the sensors detect a car ahead driving slower then the speed requested by the driver, it adapts the regulated speed in order to keep a certain safety distance. If the traffic ahead vanishes, the speed requested by the driver is resumed. At any time the driver can override the cruise control, either by explicitly deactivating it, or by pushing the brake.

The example specifies the *control* component of the ICC, by a *process class* (the outer box). The processes of a process class have uniform interfaces to the environment (given by the schemas tagged with *PORT*), an internal data state (the schemas tagged with *DATA*), and a behavior (given by a Statechart). Process classes can be instantiated by aggregation in other classes. The entire system specified is given as a dedicated instance of top-level class. (Process instantiation is not shown in the example, but is generally described by Statemates activity charts.)

In the example, the port *Inputs* provides sensors for the state of the brake pedal, the control lever of the ICC, the current speed of the car, and distance and relative acceleration of a car driving ahead. The port *Outputs* provides the actuators, describing the *nominalSpeed* which a conventional speed regulator should hold ($nominalSpeed = 0$ indicates that regulation shall be off).

The upper chart controls whether the cruise control is activated or not, whereas the lower chart is responsible for choosing the nominal speed. Either the speed

$LEVERPOS ::= idle \mid increase \mid decrease \mid resume \mid off$
$BRAKEPOS ::= notpressed \mid pressed$

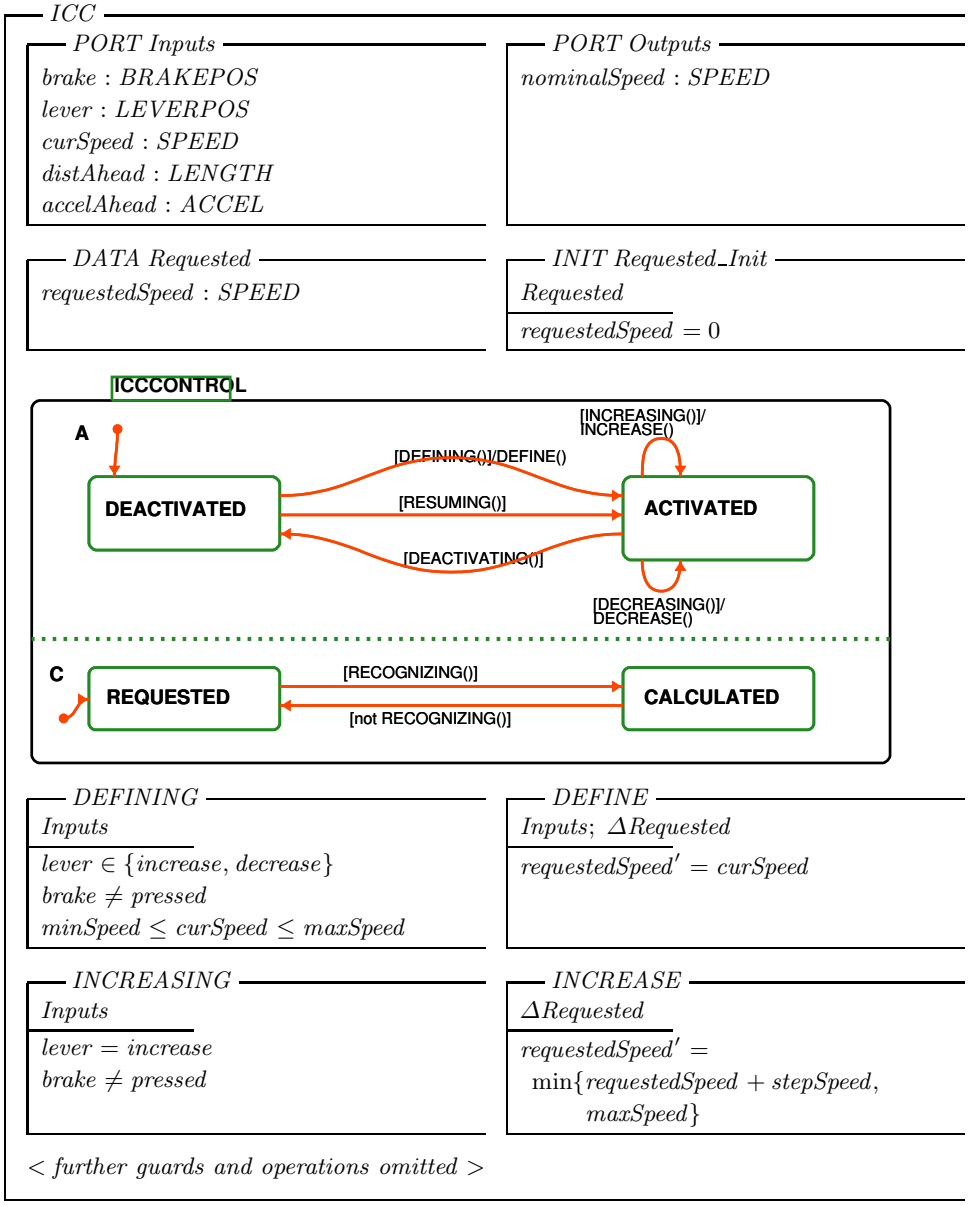


Fig. 1. $\mu\mathcal{SZ}$ example: intelligent cruise control

requested by the driver (*REQUESTED*) is used or the speed of a car ahead (*CALCULATED*).

Conditions on the data space which enable transitions and operations to be performed if a transition fires are described by a set of conventionally Z schemas, referred to from the Statecharts. The figure shows only some of these schemas, which model the activation and configuration of the ICC by the driver. In state

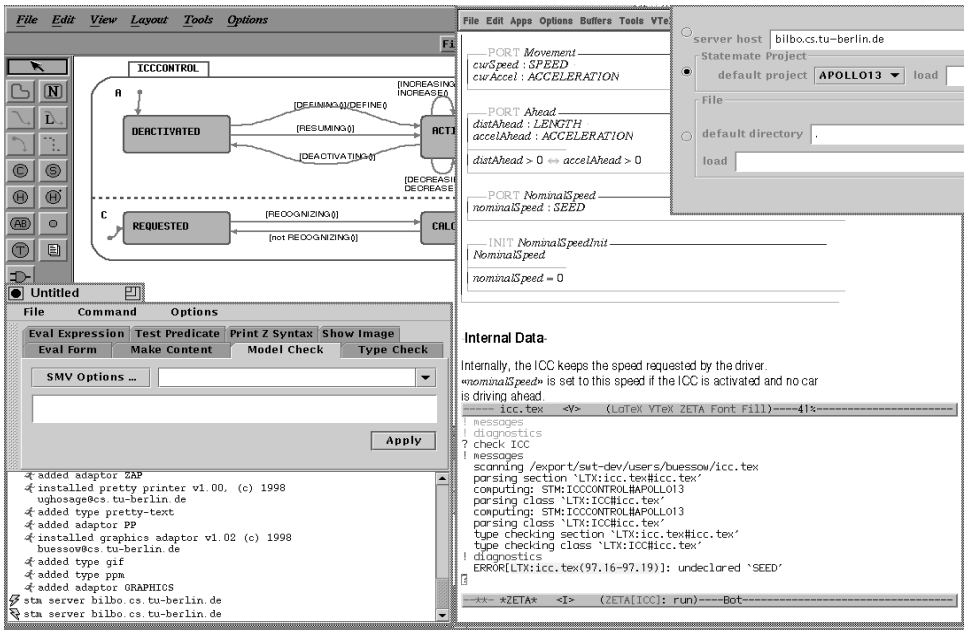


Fig. 2. A sample ZETA session

DEACTIVATED, the driver can push the lever in direction *increase* or *decrease* in order to activate the ICC, thereby setting the requested speed to the current speed. In state *ACTIVATED*, if the driver hits the lever to e.g. position *increase*, the requested speed is incremented by a certain amount.

2.2 A ZETA Session

The screenshot in Fig. 2 shows a session with the ZETA tool environment. In the upper left corner, a Statechart is edited with the Statemate tool. To the right, a $\mu\mathcal{SZ}$ specification is edited with the XEmacs editor, which has been extended by visualization support for Z and $\mu\mathcal{SZ}$. In the lower left corner, the ZETA session commander is shown, which allows to start analyzers and displays the session protocol. At the right top, a dialog of the ZETA commander for configuring the connection to the Statemate tool is active. In general, if the user modifies the Statechart under Statemate, the ZETA environment will automatically import the new version of the chart, and invalidate the results of tool chains based on the older version of the chart.

The most important analyzing techniques and transformation currently available in the ZETA environment are the following:

- parsing and type checking of $\mu\mathcal{SZ}$ specifications (since plugins such as Statecharts are translated to the $\mu\mathcal{SZ}$ core, this includes the checking of Statchart transitions).
- deduction on certain aspects of a Z/$\mu\mathcal{SZ}$ specification, for example derivation of pre-conditions, consistency of schemas, existence of initial states of $\mu\mathcal{SZ}$ classes, completeness of transitions, derivation of test-cases for operation schemas. Deduction is based on the $\mathcal{HOL}$-$\mathcal{Z}$/Isabelle tool [20].
- model-checking of temporal properties of $\mu\mathcal{SZ}$ class' behavior. Model checking is based on the SMV tool [24].

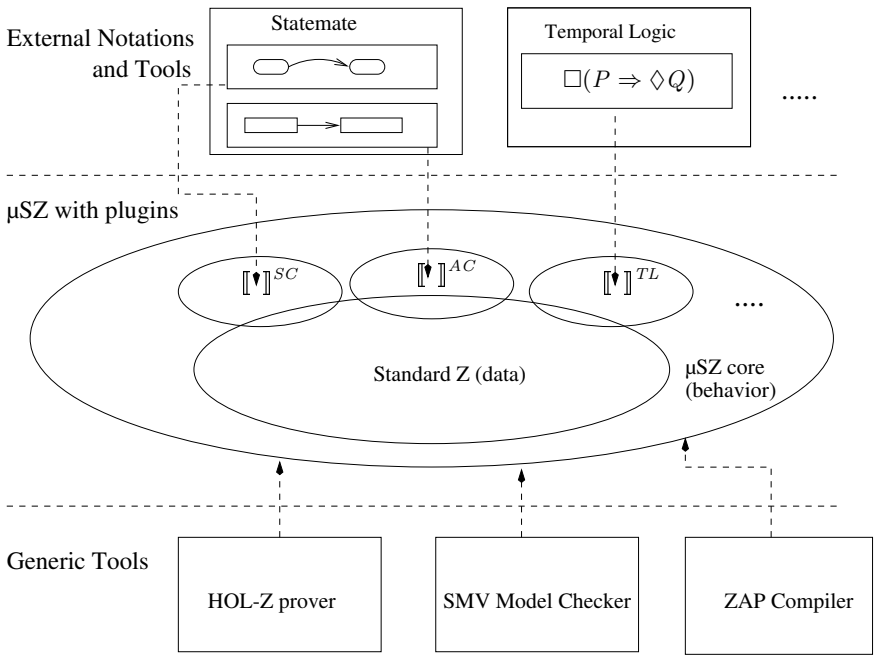


Fig. 3. The $\mu\mathcal{SZ}$ language integration framework

- compilation of a subset of $\mu\mathcal{SZ}$ and Z to executable code, based on the ZAP tool[2].
- Generation of documentation from the $\mu\mathcal{SZ}$ specification document (with automatic inclusion of the pictures from Statemate)

In the following, we present the technical concepts, underlaying of the notation $\mu\mathcal{SZ}$ and the tool environment ZETA.

3 The $\mu\mathcal{SZ}$ Notation

The formalism $\mu\mathcal{SZ}$ is based on a *core* which is independent of notations such as Statecharts or temporal logic as used for behavioral specification in ESPRESS. External notations are integrated by *plugins* which are defined by a *syntactic translation* into the $\mu\mathcal{SZ}$ core.

This is illustrated in Fig. 3. The foundation of $\mu\mathcal{SZ}$ is provided by the Standard Z notation[27], which may be also used for user-level specification of the data part. The $\mu\mathcal{SZ}$ core is a conservative extension to Z which adds the notion of *process classes*, as sketched in Sect. 2. Such classes describe concurrent, synchronous processes. External notations are integrated by translations into the core, denoted by $[\![_]\!]^{notation}$ in Fig. 3, which may target as well the data as the behavioral part of a process class. Tools which support the $\mu\mathcal{SZ}$ core can be reused to a large extent for the integrated notation.

The $\mu\mathcal{SZ}$ core provides the description of the *shared* and *private* data-space of a process class in Z, and assigns a process class a *trace of bindings* of the data-space as its model. Prior to trace construction, the data-space is extended by information to

[2] ZAP is an experimental compiler for the higher-order functional subset of Z-based specifications whose concepts have not been published yet.

model *concurrent access* to its variables. To this end, each variable of the data-space is equipped with a *lock*, which can be acquired by concurrently executing activities. The locks allow to resolve *racing*, the situation where two parallel activities write to the same variable at the same execution step, in which case one of the write operations wins. Racing is a prototypical situation of concurrent resource allocation in the synchronous approach, and can be used to model more high-level communication facilities, such as channels and service-access points (e.g. remote procedure calls).

To integrate a new formalism, it is necessary to define a translation of this formalism into the $\mu\mathcal{SZ}$ core. For the Statemate languages, this is shown in Sect. 4. For integration into the ZETA tool environment an *model source adaptor* that does the translation has to be implemented. The ZETA tool environment and its adaptor concept is described in Sect. 5. Whether an integration of a formalism is possible depends on whether a translation can be found. To what extent generic $\mu\mathcal{SZ}$ tools can be reused depends on the $\mu\mathcal{SZ}$ core definitions resulting from this translation. The SMV analysis adaptor for model-checking, for example, needs the definition of the traces in terms of a transition relation. Furthermore, it only handles an executable and finite subset of Z used for the data-space of a process class.

Below, we provide the semantic definition of the core of $\mu\mathcal{SZ}$, including the construction of the data-space of a process class by Standard Z, the lifting of the data-space to concurrency by variable locks, and the trace-semantics of a process class.

3.1 The Data-Space

The core of $\mu\mathcal{SZ}$ provides constructs to specify the data-space of a process class in a way quite similar to the state-oriented style found in conventional Z specifications [26]: schemas are used for specifying the data-space itself, conditions on the data-space, and operations on the data-space. However, whereas Z does not define explicitly how a model of the specified system (process) is constructed from the individual schemas, $\mu\mathcal{SZ}$ does so by assigning certain *roles* to schemas. The roles are the following:

- *The PORT role:* The variables declared in these schemas constitute the *public* data, i.e. the interface of a process class. These variables can be read and/or written by other processes. The axioms of *PORT* schemas describe invariants over the values of the public variables which hold continuously for all the process class' instances. In the example in Fig. 1, *Outputs* and *Inputs* are port-schemas.
- *The DATA role:* The variables declared in these schemas constitute the *private* data of a process class. In the example, *Requested* is a data-schema.
- *The INIT Role:* The variables of an *INIT* schema have to be already part of the data-space. The axioms of the *INIT* specify the values of the variables mentioned in its declaration at the beginning of the lifetime of a process. In the example, *Requested_Init* is an init-schema.

Each process class has implicit definitions of schemas for summarizing all port and data variables and initialization states. Let P_i $(i = 1 \ldots n)$ be the schemas with role *PORT*, D_i $(i = 1 \ldots m)$ with role *DATA*, and I_i $(i = 1 \ldots k)$ with role *INIT*:

$$
\begin{array}{ll}
\mathsf{PORT} & \widehat{=}\ P_1 \wedge \ldots \wedge P_n \\
\mathsf{DATA} & \widehat{=}\ D_1 \wedge \ldots \wedge D_m \\
\mathsf{DATASPACE} & \widehat{=}\ \mathsf{DATA} \wedge \mathsf{PORT} \\
\mathsf{INIT} & \widehat{=}\ [\mathsf{DATASPACE} \mid I_1 \wedge \ldots \wedge I_k]
\end{array}
$$

These definitions have the following consequences. A variable can be declared several times in different data- or port-schemas. In this case, the Z context conditions demand that the variable is always declared with the same type. If a variable is declared in a port- and a data-schema, it is shared. More precisely, the variable is exported by each port it is declared in. The definitions also demand that init-schema do not declare new variables. The schema DATASPACE denotes the complete data-space of a process class.

3.2 Variable Locks

Variables of the data-space of a process class can, in general, be written concurrently by processes in the environment which share the same port variables or by parallel activities of the class itself (recall e.g. the parallel chart in Sect. 2). We use *variable locks* to resolve the situation of *racing*, where several activities write at the same (synchronous) time to a variable.

In order to realize locks in the model we need to distinguish concurrent activities in an abstract way. We identify concurrent activities by *places*. Each access of a variable is assigned via its lock a place currently accessing the variable. There is a special place *none*, expressing that no activity is accessing the variable. In this case, a default behavior is applied, e.g. a persistent variable keeps its value or a volatile variable (event) is reset (however, we only tackle persistent variables in this paper). The type of places is defined as follows:

$$[PLACE] \qquad \mid none : PLACE$$

The schema $\overline{\mathsf{DATASPACE}}$ denotes the data-space of a process class lifted to the data-space with locking variables. It also contains a variable *here* which identifies a (concurrent, synchronous) operation performed by this process class on its data-space (the usage of *here* will become clearer below). Let DATASPACE have the signature $[x_1 : X_1;\ \ldots;\ x_n : X_n]$, then

$$\overline{\mathsf{DATASPACE}} \mathrel{\widehat=} \mathsf{DATASPACE} \land [\, x_lock_1 : PLACE;\ \ldots;\ x_lock_n : PLACE;\ here : \mathbb{F}(PLACE \setminus \{\, none \,\})]$$

As will be seen in the next section, the semantic model of a process-class is a set of traces of bindings to the schema $\overline{\mathsf{DATASPACE}}$.

We can now describe what the concurrent execution of Z operations on the data-space of a class shall mean. Note that this definition is not actually part of the core of $\mu\mathcal{SZ}$ – it is just a semantic tool provided for language-plugins such as Statecharts (Sect. 4).

Let *Op* be some Z operation with a sub-signature of ΔDATASPACE. Let $Y' = \{y'_1, \ldots, y'_n\}$ be the primed variables of the signature of *Op*, and let $\{z'_1, \ldots, z'_m\}$ the primed variables of ΔDATASPACE which are *not* in Y'. We say that the y'_is are *written* by *Op*[3]. $\overline{Op}$ denotes the lifting of *Op* to a concurrent activity, and is defined as follows:

[3] Note that the fact whether an operation "writes" a variable or not is syntactically derived from its occurrence in the signature – a semantic definition of writing does not make sense because operations may be loose, in general. This syntactic characterization is adequate for $\mu\mathcal{SZ}$, since *PORT* and *DATA* schemas support selective use of primed variables in operations; see e.g. the schema *DEFINE* in Fig. 1.

$\overline{Op}$

$\Delta\overline{\mathsf{DATASPACE}}$

$\#here' = 1$
$\{\, y_lock'_1, \ldots, y_lock'_n \,\} \neq \{\, none \,\}$
$here' \cap \{\, y_lock'_1, \ldots, y_lock'_n \,\} \neq \varnothing \Rightarrow here' = \{\, y_lock'_1, \ldots, y_lock'_n \,\} \wedge Op$
$here' \cap \{\, z_lock'_1, \ldots, z_lock'_m \,\} = \varnothing$

The lifted operation is assiged exactly one place ($\#here' = 1$). We do not assign fixed places to operations. In fact, the place of an operation can change from step to step: we only demand (as shown below) that parallel executing operations have disjoint places. $\overline{Op}$ enforeces that at least one of the variables y'_i has to be written by an activity (if not by Op itself then by another concurrent activity). $\overline{Op}$ either locks all variables y'_i or none. Since $here'$ has exactly one element, all $y_lock'_i$ have the same values, if the operations locks them. If $\overline{Op}$ acquires the lock on all variables, the effect of the transformation of Op on the data-space takes place. Furthermore, $\overline{Op}$ never locks the variables z_i.

The following example illustrates the lifting of an operation A, after some simplifications on $\overline{A}$:

A

$x, x' : \mathbb{N}$

$x' = x + 1$

$\overline{A}$

$x, x' : \mathbb{N};\ x_lock, x_lock' : PLACE$
$here : \mathbb{P}\, PLACE \setminus \{\, none \,\}$

$\#here' = 1;\ x_lock' \neq none$
$x_lock' \in here' \Rightarrow A$

What is the role of the variable *here* in this construction, and why is it a set of places? First, *here* is given as a variable of the data-space since we do not want to assign "fixed" places to the activity being executed. As will be seen in the remaining parts of this paper, we never actually need to assign some constant place value to an activity; we keep the places loose, just demanding disjointness of certain place sets. This increases compositionality of the approach. Second, *here* is given as a *set* of places in order to allow to describe the behavior of a class by step-wise *composition* of simple operations which use one place (as $\overline{Op}$) into more complex ones.

We now define the *parallel composition* of two operations Op_1 and Op_2 as a concurrent activity as follows. The places of the composed activity are partitioned by the places assigned to sub-activities $\overline{Op_1}$ and $\overline{Op_2}$:

$\overline{Op_1} \| \overline{Op_2}$

$\Delta\overline{\mathsf{DATASPACE}}$

$\exists\, h_1, h_2 : \mathbb{P}\, PLACE \bullet \langle h_1, h_2 \rangle$ partition $here' \wedge \overline{Op_1}[h_1/here'] \wedge \overline{Op_2}[h_2/here']$

Consider, for example, the operation A defined above and a second operation $B \mathrel{\widehat{=}} [x, x' : \mathbb{Z} \mid x' = x + 2]$ to be concurrently executed. The lifting to a parallel activity yields (after some simplification):

$\overline{A}\|\overline{B}$

$x, x' : \mathbb{Z};\ x_lock, x_lock' : PLACE$
$here;\ here' : \mathbb{P}\ PLACE \setminus \{\ none\ \}$

$\exists\, h_1, h_2 : PLACE \bullet \langle h_1, h_2\rangle$ partition $here' \land \{\ x_lock'\ \} \neq \{\ none\ \} \land$
$(x_lock' \in h_1 \Rightarrow A) \land (x_lock' \in h_2 \Rightarrow B)$

The schema $\overline{A}\|\overline{B}$ coordinates the parallel execution of A and B, still allowing $\overline{A}\|\overline{B}$ to compete with other activities accessing x'. Write access to x can be limited to A and B by the additional axiom: $x_lock \in here$. This will be used in the following, to grant processes exclusive access to their private variables.

For further applications of locks, see the Statechart-plugin into $\mu\mathcal{SZ}$ described in Sect. 4.

3.3 Model of a Process Class and Plugin Interface

The semantics of a process class is defined as a set of *traces* (infinite sequences) over the lifted $\overline{\mathsf{DATASPACE}}$. The traces are given by a constant classTraces which is implicitly defined in each process class. In this case, the set of places *here* denotes places of the process. Furthermore, there is a constant dataTraces which specifies the traces which are admissible according to the data-space of the class. The initial states of these traces are defined to be initial states of the process ($t\,1 \in \mathsf{INIT}$), variables that are not written (i.e. not locked) during a step, keep their values, and write access to private variables is restricted to this very process. If $x_1 \ldots x_n$ are the variables of the process' data-space and $y_1 \ldots y_m$ the private variables among them ($\{\ y_1, \ldots, y_m\ \} \subseteq \{\ x_1, \ldots, x_n\ \}$) then dataTraces and classTraces are defined as follows:

$\mathsf{trace}\, X == \mathbb{N} \longrightarrow X$

$\mathsf{classTraces}, \mathsf{dataTraces} : \mathbb{P}(\mathsf{trace}\, \overline{\mathsf{DATASPACE}})$

$\mathsf{dataTraces} =$
$\quad \{t : \mathsf{trace}\, \overline{\mathsf{DATASPACE}} \mid$
$\qquad t\,1 \in \mathsf{INIT} \land$
$\qquad (\forall\, i : \mathsf{dom}\, t \bullet$
$\qquad\quad (t(i+1)).x_lock_1 = none \Rightarrow (t(i+1)).x_1 = (t\,i).x_1$
$\qquad\qquad \land \ldots \land$
$\qquad\quad (t(i+1)).x_lock_n = none \Rightarrow (t(i+1)).x_n = (t\,i).x_n \land$
$\qquad\quad (t\,i).y_lock_1 \in (t\,i).here$
$\qquad\qquad \land \ldots \land$
$\qquad\quad (t\,i).y_lock_m \in (t\,i).here)\}$
$\mathsf{classTraces} \subseteq \mathsf{dataTraces}$

Language plugins effecting behavior are syntactically translated into predicates over the variable classTraces, further constraining or fixing the value of classTraces.

Traces over the data-space are an adequate model for the semantics of synchronous languages. But for tool support, such as model checking or execution, they have some disadvantages. Implementation of these techniques is much easier if the semantics is given as a state transition relation. We provide the function asTraces, translating a state transition relation into the set of traces that are reachable from the initial states by applying the transition relation. A plugin that defines the process' semantics by a state transition relation T can define classTraces with the axiom $\mathsf{classTraces} = \mathsf{asTraces}\ T$. A tool can then use the transition relation T rather then the class traces themself.

$$\mathsf{asTraces} == \lambda\, trans : \mathbb{P}(\Delta\overline{\mathsf{DATASPACE}}) \bullet \{ t : \mathsf{dataTraces} \mid \forall\, i : \mathsf{dom}\, t \bullet (t\, i, t(i+1)) \in \{\, trans \bullet (\theta\overline{\mathsf{DATASPACE}}, \theta\overline{\mathsf{DATASPACE}}') \,\} \}$$

Note that the variable *trans* denotes a schema, and can be used in declaration position as proposed in the upcoming Z standard [27].

4 The Statemate Plugins to $\mu\mathcal{SZ}$

In this section we show how the Statemate languages, Statecharts and activity charts, are integrated as plugins into the $\mu\mathcal{SZ}$ core.

4.1 Statecharts

The Statecharts-plugin is given as a translation into a state-transition relation. Here, we only describe the central elements of the translation but skip more intricate details. We assume that the reader is familiar with Statecharts and their semantics. For a detailed discussion of the semantics, we refer to [14]. Timing aspects and super steps are omitted here. The semantics, thus, reflects what is called *synchronous* semantics in Statemate terminology. We illustrate the translation by the example given in Fig. 4.

$$Basic, Xor, And : \mathbb{P}\, STATE$$
$$root : STATE$$
$$parent : (STATE \setminus \{\, root \,\}) \longrightarrow (Xor \cup And)$$
$$\langle Basic, Xor, And \rangle \ \mathsf{partition}\ STATE$$

$$DATA\ \mathsf{Config}$$
$$\mathbf{c} : Xor \longrightarrow STATE$$
$$\mathbf{c} \subseteq parent^{\sim}$$

At first, the translation introduces a given type *STATE* representing the Statecharts states. *STATE* is partitioned into the sets of basic-, xor-, and and-states. The *parent* relation assigns each state but the root state its parent state. Figure 4 instantiates these definitions for a concrete Statechart.

The *active states* of a Statechart are modeled with a variable, the *configuration* $\mathbf{c} \in Xor \longrightarrow STATE$, which becomes part of the data-space (Fig. 4). $\mathbf{c}$ assigns at each execution step each xor-state one of its substates—its *current* substate. A state S is active iff. (1) it is the root state, (2) its parent is an and-state and is active, or (3) its parent is an active xor-state and S is the current state of its parent, i.e. $\mathbf{c}(parent\ S) = S$.

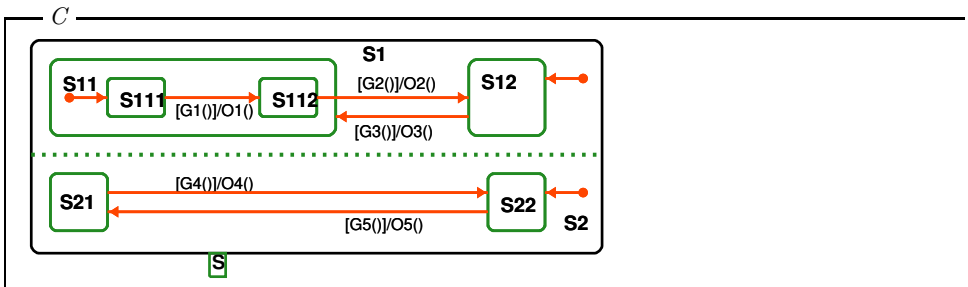
The Statechart-semantics is now defined as a state transition relation over the data-space, including the configuration. The translation function $\mathcal{S}[\![_]\!] : STATE \longrightarrow \Delta\overline{\mathsf{DATASPACE}}$ yields for each state its transition relation (a Z schema operation over $\Delta\overline{\mathsf{DATASPACE}}$). Figure 4 shows how the transition relations are computed for the example, i.e. the schema T represents the transition relation of the root state S.

$$\mathcal{S}[\![S]\!] \mathrel{\widehat{=}} (S \in Basic \Rightarrow \mathcal{S}_{bas}[\![S]\!]) \wedge (S \in And \Rightarrow \mathcal{S}_{and}[\![S]\!]) \wedge (S \in Xor \Rightarrow \mathcal{S}_{xor}[\![S]\!])$$

$$\mathcal{S}_{bas}[\![S]\!]$$
$$\Delta\overline{\mathsf{DATASPACE}}$$
$$here' = \varnothing$$

$$\mathcal{S}_{and}[\![S]\!]$$
$$\Delta\overline{\mathsf{DATASPACE}}$$
$$\mathcal{S}[\![S_1]\!] \parallel \ \ldots\ \parallel \mathcal{S}[\![S_n]\!]$$

The most interesting part of the definition of the transition-relation is the translation for xor-states, $\mathcal{S}_{xor}[\![S]\!]$. That translation is, depending on the supported features, as interlevel transitions, connectors, history, etc., quite complex. Here, we



C

$STATE ::= S \mid S1 \mid S11 \mid S111 \mid S112 \mid S12 \mid S2 \mid S21 \mid S22$

$Basic, Xor, And : \mathbb{P}\, STATE$
$root : STATE$
$parent : (STATE \setminus \{ root \}) \longrightarrow (Xor \cup And)$

$\langle Basic, Xor, And \rangle$ partition $STATE$
$Basic = \{ S111, S112, S12, S21, S22 \}$
$Xor = \{ S1, S11, S2 \}$
$And = \{ S \}$
$root = S$
$parent = \{ S111 \mapsto S11, S112 \mapsto S11, S11 \mapsto S1, \ldots \}$

DATA Config

$\mathbf{c} : Xor \longrightarrow STATE$

$Basic \mathrel{\widehat{=}} [\Delta\overline{\mathsf{DATASPACE}} \mid here' = \varnothing]$
$T111 \mathrel{\widehat{=}} Basic;\ T112 \mathrel{\widehat{=}} Basic;\ T12 \mathrel{\widehat{=}} Basic;\ T21 \mathrel{\widehat{=}} Basic;\ T22 \mathrel{\widehat{=}} Basic;$
$\gamma_1 \mathrel{\widehat{=}} [\overline{\mathsf{DATASPACE}} \mid \mathbf{c}(S11) = S111 \wedge G1]$
$\alpha_1 \mathrel{\widehat{=}} [\Delta\overline{\mathsf{DATASPACE}} \mid \mathbf{c}'(S11) = S112 \wedge \overline{O1}]$
$\gamma_2 \mathrel{\widehat{=}} [\overline{\mathsf{DATASPACE}} \mid \mathbf{c}(S1) = S(11) \wedge \mathbf{c}(S11) = S112 \wedge G2]$
$\alpha_2 \mathrel{\widehat{=}} [\Delta\overline{\mathsf{DATASPACE}} \mid \mathbf{c}'(S1) = S12 \wedge \overline{O2}]$
$\gamma_3 \mathrel{\widehat{=}} [\overline{\mathsf{DATASPACE}} \mid \mathbf{c}(S1) = S(12) \wedge G3]$
$\alpha_3 \mathrel{\widehat{=}} [\Delta\overline{\mathsf{DATASPACE}} \mid \mathbf{c}'(S1) = S11 \wedge \mathbf{c}'(S11) = S111 \wedge \overline{O3}]$
$\gamma_4 \mathrel{\widehat{=}} \ldots$

T11

$\Delta\overline{\mathsf{DATASPACE}}$

if γ_1
  then $\gamma_1 \wedge \alpha_1$
  else $\mathbf{c}'(S11) = \mathbf{c}(S11) \wedge$
    $(\mathbf{c}(S11) = S111 \Rightarrow T111) \wedge$
    $(\mathbf{c}(S11) = S112 \Rightarrow T112)$

T1

$\Delta\overline{\mathsf{DATASPACE}}$

if $\gamma_2 \vee \gamma_3$
  then $(\gamma_2 \wedge \alpha_2) \vee (\gamma_3 \wedge \alpha_3)$
  else $\mathbf{c}'(S1) = \mathbf{c}(S1) \wedge$
    $(\mathbf{c}(S1) = S11 \Rightarrow T11) \wedge$
    $(\mathbf{c}(S1) = S12 \Rightarrow T12)$

T

$\Delta\overline{\mathsf{DATASPACE}}$

$T1 \| T2$

classTraces = asTraces T

Fig. 4. Example Statechart and its translation

do not discuss simplification of complicated transition arrows. For each transition, we know the states it leaves and the states it enters (e.g. the example transition labeled $[G2()]/O2()$ exits $S111$ and $S11$ and enters $S12$). We augment the guards by demanding that the exited states are active (i.e. $\gamma_2 \equiv \mathbf{c}(S1) = S11 \wedge \mathbf{c}(S11) = S112 \wedge G0$). The augmented actions demand that the entered states (including default completions) are active in the post state (i.e. $\alpha_2 \equiv \mathbf{c}'(S1) = S12 \wedge \overline{O2}$).

To handle interlevel-transitions we consider the *arena states* of the transitions. The arena state is the state, the transitions crosses but neither enters nor leaves. In the state hierarchy, it is the highest state the transition crosses. Arena states are always xor-states. The arena of the transition labeled $[G2()]/O2()$ in the example is, e.g., $S1$. If a state is the arena state of a transition, we say, the other way round that the transition is an *arena transition* of the state. An xor-state either executes one of its arena-transitions or behaves as its active substate.

For an xor-state S with m arena transition, where the guards (including active source states) are given by $\gamma_1, \ldots, \gamma_m$ and the actions (including state change) by $\alpha_1, \ldots, \alpha_m$ and substates $S_1 \ldots S_n$ we have:

$\mathcal{S}_{xor}[\![S]\!]$

$\Delta\overline{\mathsf{DATASPACE}}$

$$\begin{array}{l}\textbf{if } \gamma_1 \vee \ldots \vee \gamma_m \\ \quad \textbf{then} (\gamma_1 \wedge \alpha_1) \vee \ldots \vee (\gamma_m \wedge \alpha_m) \\ \quad \textbf{else } \mathbf{c}'(S) = \mathbf{c}(S) \wedge \\ \qquad (\mathbf{c}(S) = S_1 \Rightarrow \mathcal{S}[\![S_1]\!]) \wedge \ldots \wedge (\mathbf{c}(S) = S_n \Rightarrow \mathcal{S}[\![S_n]\!])\end{array}$$

The schemas $T1$ and $T11$ in Fig. 4 illustrate the construction of the transition relation for xor-states. For the example Statechart, the translation yields a Z schema for each state as shown in Fig. 4. The $\mathsf{classTraces}$ are fixed by the transition relation T, i.e. the transition relation of the root state S.

The traces of the class are finally defined to be the traces induced by the transition relation of the state *root*:

$$\mathsf{classTraces} = \mathit{asTraces}\, \mathcal{S}[\![\mathit{root}]\!]$$

4.2 Activity Charts

Similiar to Statecharts, Statemate's activity charts are translated into the $\mu\mathcal{SZ}$ core. Consider the activity chart contained in class A in Fig. 5. The class *aggregates* the processes P and Q. These two processes share the variable x over the port X. The variable y of Q is propagated to the environment through the port Y.

The activity chart is translated into the core as shown in Fig. 6. A gets an additional port Y to export the variable y. Generally, variables that are bound by information flows are identified in each step. If a variable is not exported, its lock variable is restricted, in order to express that no other activity accesses the variable. The places of the aggregated processes partition the set of places of the aggregating process. Processes that can not access certain variable (P can not access y) are excluded from the lock variables of these variables ($y_lock \notin (t_P\ i).here$).

5 The Tool Environment ZeTa

One of the central ideas of using combined notations is to *reuse* existing tools for the individual constituting notations – on the one hand to allow users to continue working with the tools they are acquainted with (in the context of Espress, e.g.,

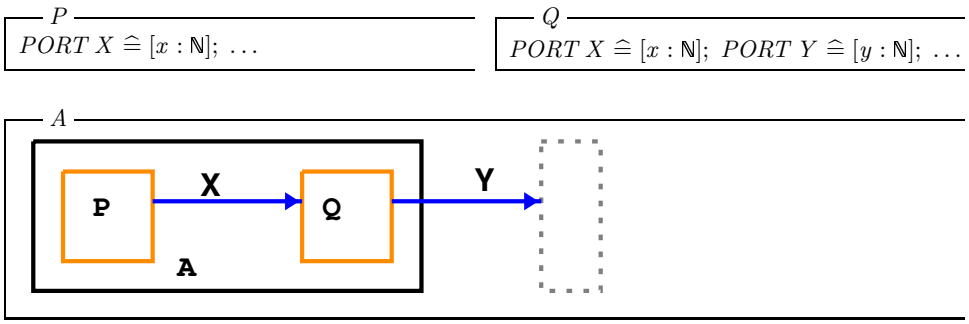


Fig. 5. Example activity chart

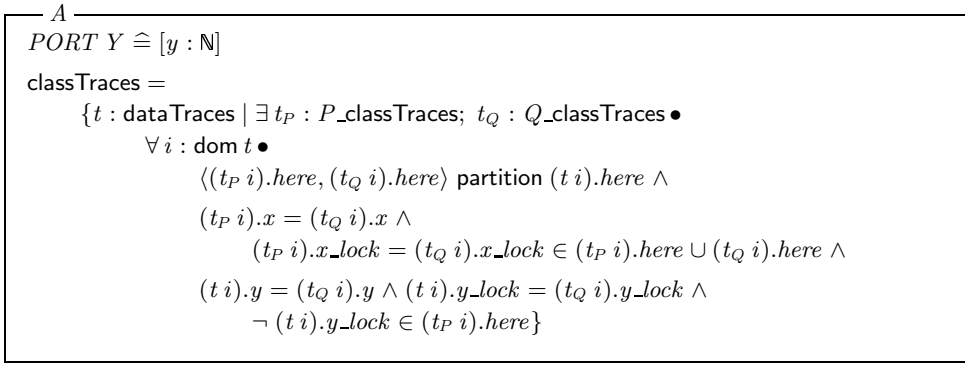


Fig. 6. Translation of the example activity chart

the Statemate tool, which is relatively widely spread in the industry for embedded systems), on the other hand since it is not realistic to re-implement certain tools from scratch in the desired quality.

The purpose of the ZETA tool environment as developed in ESPRESS is to provide an *open* framework for the integration of existing and newly written tools. These tools directly support the notations which are semantically integrated as plugins in the $\mu\mathcal{SZ}$ notation. ZETA also supports generic tools based on the Z or $\mu\mathcal{SZ}$ language (type checkers, provers, model checkers, compilers) which can process the integrated notations by their mapping to Z and/or by proprietary mappings, and other tools e.g. for generating documentation.

5.1 Integrated Specification Documents

The "adhesion" of integration in the ZETA environment is an *integrated specification document* (ISD). In most cases this is a $\mu\mathcal{SZ}$ specification, but in general an ISD needs not to contain formal parts.

An ISD is given in a markup language (currently, LaTeX) and contains references to fragments of specification contributed by external tools. The references are described by *model source locators* (MSL), a concept comparable to URLs. An MSL identifies the tool that is to provide the (sub-)model and a tool specific identifier for the desired portion. This is illustrated in Fig. 7. (Beside the external tool State-

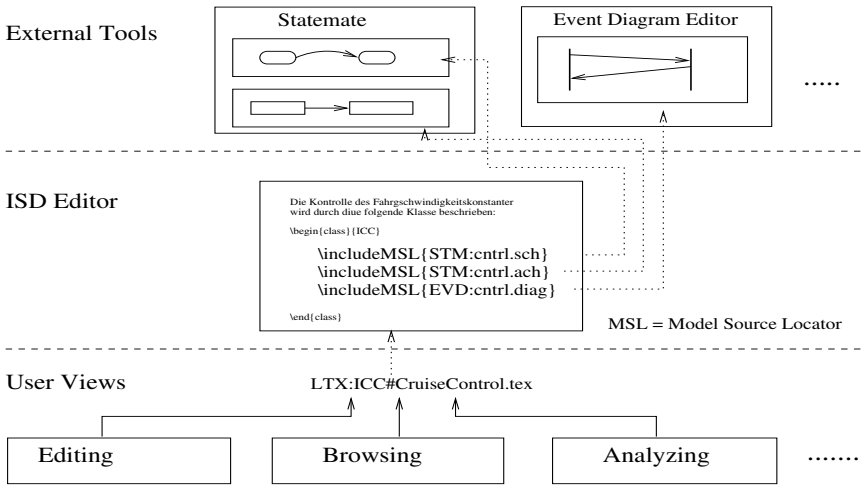


Fig. 7. Integrated Specification Documents

mate, a tool for editing event diagrams as a graphical frontend for temporal logics is shown for illustration purposes.)

The major difference between the concept of URLs and of MSLs is that the latter are interpreted in different ways, depending on the operation to be performed on an ISD. For example, if an ISD is browsed, its MSLs are resolved to encapsulated postscript. If it is analyzed, the MSLs may be resolved to an $\mu\mathcal{SZ}$ core abstract syntax term which is inserted in the enclosing process class (depending on the kind of analysis and MSL).

The ISD itself is addressed by an MSL as well: if the ISD is given in the LaTeX-markup, then such an MSL looks e.g. like `LTX:ICC#CruiseControl.tex`. Hereby, `LTX` denotes the tool providing this MSL (the LaTeX-System), `ICC` denotes a class (here the cruise control), and `CruiseControl.tex` a file where this class is found.

5.2 Content and Adaptors

The basic operation provided by the ZETA integration environment is to *query* the content of an MSL. This query is parameterized by the *type* of the desired content. Typical types are "gif", "encapsulated postscript", "parsed abstract syntax", "type checked abstract syntax", "plugin abstract syntax", and so on. Types might become more complex containing further selective information, e.g. "the precondition of schema S".

The ZETA environment behaves as a broker between queries and tools which have registered in the environment to be able to compute content of a specific type for MSLs. Once a tool is contacted to compute a particular content, it may recursively query the environment for further content. For example, when a type checker is contacted to compute the "checked abstract syntax" of an $\mu\mathcal{SZ}$ class, it first queries the "parsed abstract syntax". If this syntax contains MSLs referring to external notations, it next queries the "plugin abstract syntax", then merges the syntaxes, and finally performs it type check operation. This way, tool chains are realized.

The basic model of ZETA is comparable to a traditional Unix `make`, but it is more powerful and reliable, since it is embedded in the strongly-typed Java programming environment:

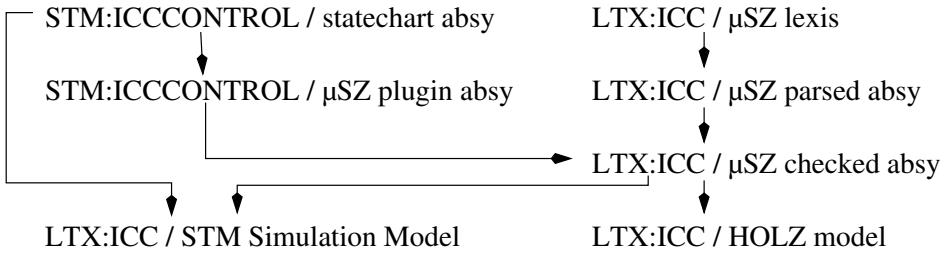


Fig. 8. Chains of content queries

- The content exchanged by tools via ZETA are standardized data-formats defined by Java APIs. For the $\mu\mathcal{SZ}$ language (and the Z language embedded in $\mu\mathcal{SZ}$) the abstract syntax is pregiven by ZETA, and tools on the syntax such as type inference, name environments and so on.
- A tool is represented in a ZETA session as a Java object whose instances implement the interface of so-called *tool adaptors*, and which can have state and behavior. The adaptor object may implement its functionality completely in Java, may wrap another Unix process, or may be an RMI (remote method invocation) proxy to a remotely running JVM.
- Tool adaptors may import external models into the ZETA session from other running tools, not just the file system. For instance, the Statemate adaptor imports initial content for his MSLs, `STM:<chart>`, by connecting to the data-port of a running Statemate session.
- ZETA caches content resulting from computations, thereby providing an outdating mechanism. Whereas `make` uses time-stamps of files to decide whether an initial content is outdated (and henceforth all the content derived from it), in ZETA adaptors which provide initial content may use arbitrary techniques for this purpose.
- ZETA realizes a session protocol for computations, which, among others, keeps book of diagnostics produced by computations.

5.3 ZETA and $\mu\mathcal{SZ}$

The plugins of the $\mu\mathcal{SZ}$ notation directly correspond to a certain kind of tool adaptors plugged into the ZETA environment (in fact, historically, first the ZETA plugins existed, then the $\mu\mathcal{SZ}$ plugins). These adaptors, which import an external notation into ZETA by providing initial content for MSLs, are called *model source adaptors* (MSA).

In the simplest case, an MSA directly performs the translation of the external notation to the $\mu\mathcal{SZ}$ core (as described for the Statemate languages in Sect. 4). But usually, there are intermediate steps in the way from an external notation to a $\mu\mathcal{SZ}$, and the intermediate content may be used by tools specialized for them.

This is illustrated by Fig. 8. The Statemate-MSA initially creates an abstract syntax for the Statechart denoted by the MSL `STM:ICCCONTROL`, which is in a second step converted to the $\mu\mathcal{SZ}$ core syntax of an $\mu\mathcal{SZ}$ plugin. In turn, the LaTeX-MSA initially creates an $\mu\mathcal{SZ}$ lexis for the MSL `LTX:ICC` (the file name in the MSL has been omitted), which is converted first to a parsed abstract syntax, then to a type-checked abstract syntax. Assuming that the class denoted by `LTX:ICC` refers to the MSL `STM:ICCCONTROL`, the type-checker queries for the abstract syntax of the $\mu\mathcal{SZ}$ plugin associated with this MSL, and includes it in the resulting type-checked

abstract syntax (thereby checking type-consistency of the plugin within its class context).

Now, the generation of a model for the $\mathcal{HOL}$-$\mathcal{Z}$ prover adaptor is entirely based on the checked abstract syntax, with the plugins expanded. However, in order to generate a model for the Statemate simulator, it is necessary to query the checked $\mu\mathcal{SZ}$ syntax as well as the original representation of the Statechart[4].

This example shows that, in the real world, the integration provided by $\mu\mathcal{SZ}$ plugins is indeed not always the only underlaying representation for tools. The translation of an external notation to the $\mu\mathcal{SZ}$ core is an abstraction step, where information (in a syntactical sense) is lost.

6 Related Work and Discussion

Specifying reactive systems with Z. There are several case studies where conventional Z has been used for the description of reactive systems. In [1] a comparative case study with several specification formalisms (including Z) has been carried out. The authors claim that for practical use of Z, a significant amount of structure has to be added. They conclude that Z was not a well-suited method for the specification of reactive systems.

In [25], the benefit of Z as a formalism for the specification of real-time control systems was examined. For the case study control structures like *while..do* loops were employed in addition to Z. It was stated that parallel processes are fundamental for the application area and thus must be expressed by the specification formalism.

In [12] a cabin illumination system was specified in Z. Here, concepts for resources and system cycles were introduced, but only expressed in natural language, rather then being reflected in the specification formalism itself. The interfaces of the control software are marked only in the explanatory text.

The authors of [19] point out that in conventional Z specifications, the details of when Z operations are invoked and how they interact with external interfaces are glossed over. This is identified as a serious problem. Furthermore, the authors emphasize the necessity of reuse of specification parts and conventions for the specification of real-time systems.

From the experience described above, it is obvious that concepts like parallel processes are inevitable for the description of reactive systems. These are supported by $\mu\mathcal{SZ}$'s process classes.

Combining Process Algebras with Z. An alternative approach to ours is the combination of a process-algebra (such as CSP) with axiomatic specification techniques (e.g. LOTOS) or with Z. An overview on recent work in combining process algebra with Z is given in [10]. This approach is semantically more powerful than ours, since the usual models for process algebra do represent "silent moves" resp. internal nondeterminism [17, 16], which our trace-based semantics cannot.

Beside of comparing the semantic power, it is interesting to compare the specification methodology resulting from the different approaches. In the Z+CSP approaches, data-state is modeled by special "server" processes which encapsulate the access to the data. This leads to a very strict partition between data and behavioral specification parts, which users of this technique might regard as cumbersome. In contrast, our approach collects data and behavior in one "unit" of specification, a process class. This resembles common techniques in (object-oriented) programming, but it also complicates the semantic model and thereby the formal reasoning

[4] Generating a simulation model amounts to compile code for the Z schemas referred to from transitions and bind them to the Statemate simulator. To generate the binding code, the structure of the Statechart needs to be known.

by the possibility of racing already in the date-state specification. However, there are techniques how the impossibility of racing in certain situations can be expressed by further axioms about places in the generated Z of a plugin, for the use by provers and other tools. More research into characterizing those situations is required, to help to enhance tool support.

Using Temporal Logic as a Foundation for Behavior. On alternate to working directly on traces, as the $\mu\mathcal{SZ}$ plugins do, is to use some version of temporal logic, and map the behavioral notations to terms of this logic. To some extent, this approach was sketched in [6]. In this work, we presented an interval logic, embedded in Z, using places and locks to model concurrency very similar to this work. An advantage of using temporal interval logics as a foundation for behavior is that this representation is good suited for tool support, since it is combinator-based, and thus adequate for term-based transformations. However, more research is required to make this approach workable for $\mu\mathcal{SZ}$ and ZETA, and analysis tools need to be written which actually can exploit a TL-based representation.

Tool Integration Environment. Few projects approach the task of integrating tools for formal methods and existing commercial CASE tools in all the aspects as ZETA. The approach of the UniForM Workbench [21, 22] strongly relates to ours. However, whereas our integration language Java is object oriented, UniForM employs a functional language (Haskell) for this task. Also the architecture differs: UniForM shows a greater number of general components, called managers. We implement the functionality of these managers by just one component, the broker between tool adaptors and content queries, thus reducing the number of interfaces. Other approaches like the Concurrency Factory [8] or AutoFocus [18] present more homogenous environments than ours, with most tools written from scratch.

7 Conclusion

We have presented a framework for the integration of heterogeneous notations and tools. Our framework is based on Z and combines Z for the specification of data with synchronous languages for behavioral specification. By now, we have instantiated the framework with Statecharts and activity charts as well as temporal logic (which wasn't tackled explicitely in this paper). The resulting combination was used in several case studies conducted in the ESPRESS project [9, 11, 4, 28]. Tool support for the combined specification notation is provided: we have running adaptors to import Statemate models, to type-check the combined model, for analysis with the SMV model checker and the prover $\mathcal{HOL}$-$\mathcal{Z}$/Isabelle, and for execution with ZAP.

The principles, $\mu\mathcal{SZ}$ and ZETA was designed upon, can integrate heterogeneous formal and non-formal modeling as well as analysis and transformation techniques. They provide a modular and generic framework that is designed to support transformation and analysis of specifications combined from different formalisms. The semantic concept of $\mu\mathcal{SZ}$ plugins has its tool-technical counterpart of model source adaptors in ZETA that implement the integration of custom formalisms. ZETA also supports the integration of new analysis tools, let them be generic over the $\mu\mathcal{SZ}$ core or specialized for notations. The degree of integration is not prescribed by the tool environment and, frankly, not all adaptors of the current ZETA version follow the scheme presented here (yet).

Our general approach is based on the discernment that we have to reuse and integrate established notations and tools to bring formal methods to practice. We emphasize that the objective of a tool environment for formal methods has to take advantage of existing, sophisticated CASE tools such as Statemate. We anticipate

that we have to live with "rough edges" when following this approach, since tools like Statemate are *not* designed for integration in broader environments. A major principle of our work is to grind these edges, instead of trying to avoid them by reinventing from scratch.

Acknowledgments. Many results and ideas presented in this paper stem from the broader context of the Espress project and the work of its many collaborators in Berlin. Regarding the work presented in this paper and comments on earlier drafts, we would like to especially thank Thomas Santen, Robert Geisler, Stephan Herrmann, and Markus Klar.

References

1. M. A. Ardis, J. A. Chaves, L. J. Jagadesaan, P. Mataga, C . Puchol, Mark G. Statskauskas, and J. Von Olnhausen. A framework for evaluating specification methods for reactive systems - experience report. *IEEE Transactions on Software Engineering*, 22(6):378–389, June 1996.
2. J. P. Bowen, A. Fett, and M. G. Hinchey, editors. *ZUM'98: The Z Formal Specification Notation, 11th International Conference of Z Users, Berlin, Germany, 24–26 September 1998*, volume 1493 of *Lecture Notes in Computer Science*. Springer-Verlag, 1998.
3. J. P. Bowen and J. E. Nicholls, editors. *Z User Workshop, London 1992*, Workshops in Computing. Springer-Verlag, 1993.
4. R. Büssow, R. Geisler, and M. Klar. Specifying safety-critical embedded systems with statecharts and Z: A case study. In Egidio Astesiano, editor, *Proc. of the 1st Intl. Conf. on Fundemantal Approaches to Software Engineering – FASE'98*, volume 1382 of *LNCS*, pages 71–87. Springer-Verlang, 1998.
5. Robert Büssow, Robert Geisler, Wolfgang Grieskamp, and Marcus Klar. The $\mu\mathcal{SZ}$ notation version 1.0. Technical Report 97–26, Technische Universität Berlin, Fachbereich Informatik, December 1997.
6. Robert Büssow and Wolfgang Grieskamp. Combining Z and temporal interval logics for the formalization of properties and behaviors of embedded systems. In R. K. Shyamasundar and K. Ueda, editors, *Advances in Computing Science – Asian '97*, volume 1345 of *LNCS*, pages 46–56. Springer-Verlag, 1997.
7. Robert Büssow, Wolfgang Grieskamp, Winfried Heicking, and Stephan Herrmann. An open environment for the integration of heterogeneous modelling techniques and tools. to appear, 1998.
8. R. Cleaveland, J.N. Gada, P.M. Lewis, S.A. Smolka, O. Sokolsky, and S. Zhang. The concurrency factory — practical tools for specification, simulation, verification, and implementation of concurrent systems. In *Proceedings of the DIMACS Workshop on Specification of Parallel Algorithms*, Princeton, NJ, May 1994.
9. M. Conrad and D. Wetter. An adaptive cruise control system – a case study for the industrial use of formal methods. In *Proceedings of the First Workshop on Formal Design of Safety Critical Embedded Systems*, 1997.
10. C. Fischer. How to combine Z with a process algebra. In Bowen et al. [2], pages 5–23.
11. Wolfgang Grieskamp, Maritta Heisel, and Heiko Dörr. Specifying embedded systems with statecharts and Z: An agenda for cyclic software components. In Egidio Astesiano, editor, *Proc. of the 1st Intl. Conf. on Fundemantal Approaches to Software Engineering – FASE'98*, volume 1382 of *LNCS*, pages 88–106. Springer-Verlang, 1998.
12. U. Hamer and J. Peleska. Z applied to the A330/340 CICS cabin communication system. In M. G. Hinchey and J. P. Bowen, editors, *Applications of Formal Methods*, pages 253–284. Prentice Hall International Series in Computer Science, 1995.
13. David Harel, Hagi Lachover, Amnon Naamad, Amir Pnueli, Michal Politi, Rivi Sherman, Aharon Shtull-Trauring, and Mark Trakhtenbrot. Statemate: A working environment for the development of complex reactive systems. *IEEE Transactions on Software Engineering*, 16 No. 4, April 1990.
14. David Harel and Ammon Naamad. The statemate semantics of statecharts. Technical report, The Weizmann Institute of Science, October 1995.

15. David Harel and Michael Politi. Modeling reactive systems with statecharts: The statemate approach. i-Logix Inc, Three Riverside Drive, Andover, MA 01810, USA, June 1996. Part No. D-1100-43, 6/96.
16. Matthew Hennessy. *Algebraic Theory of Processes*. MIT Press, 1988.
17. C. A. R. Hoare. *Communicating Sequential Processes*. Printice Hall, 1985.
18. F. Huber, B. Schätz, A. Schmidt, and K. Spies. AutoFocus - a tool for distributed systems specification. In *FTRTFT'96 – Formal Techniques in Real-Time and Fault-Tolerant Systems*, volume 1135 of *LNCS*, pages pp. 467–470. Springer Verlag, 1996.
19. J. C. Knight and D. M. Kienzle. Preliminary experience using Z to specify a safety-critical system. In Bowen and Nicholls [3], pages 109–118.
20. Kolyang, T. Santen, and B. Wolff. Towards structure preserving encoding of Z in HOL. Technical Report 986, Arbeitspapiere der GMD, April 1996.
21. B. Krieg-Brückner, J. Peleska, E.-R. Olderog, D. Balzer, and A. Baer. Universelle Entwicklungsumgebung für Formale Methoden (UniForM Workbench). Informatik Bericht 8/95, Universität Bremen, 1995.
22. C. Lüth, E. W. Karlsen, Kolyang, S. Westmeier, and B. Wolff. Hol-Z in the UniForM-workbench – a case study in tool integration for Z. In Bowen et al. [2], pages 116–134.
23. Zohar Manna and Amir Pnueli. *The Temporal Logic of Reactive and Concurrent Systems – Specification –*. Springer-Verlag, 1992.
24. Kenneth L. McMillan. *Symbolic Model Checking*. Kluwer Academic Publishers, 1993.
25. A. R. Ruddle. Formal methods in the specification of real-time, safety-critical control systems. In Bowen and Nicholls [3], pages 131–146.
26. J. M. Spivey. *The Z Notation: A Reference Manual*. Prentice Hall International Series in Computer Science, 2nd edition, 1992.
27. I. Toyn. Innovations in standard Z notation. In Bowen et al. [2], pages 193–213.
28. Kirsten Winter, Thomas Santen, and Maritta Heisel. An agenda for event-driven software components with complex data models. In W. Ehrenberger, editor, *Proceedings of the 16th International Conference on Computer Safety, Reliablity and Security (SAFECOMP)*, LNCS 1516, pages 16–31. Springer-Verlag, 1998.